

Taller — API REST con JDBC y PostgreSQL

Objetivo

Desarrollar una API REST utilizando:

- Spring Boot
- JDBC Manual
- PostgreSQL
- Arquitectura por capas
- Services
- Controllers
- Postman
- Manejo de excepciones
- Logs

Todo el proyecto debe funcionar utilizando SQL con JDBC

Requisitos Previos

El estudiante debe tener instalado:

- Java JDK
 - Eclipse
 - PostgreSQL
 - pgAdmin
 - Postman
 - Git
 - GitHub
-

Parte 1 — Crear el Proyecto

Crear un nuevo proyecto Spring Boot

Ingresar a:

<https://start.spring.io/>

Configuración

Campo	Valor
Project	Maven
Language	Java
Group	com.krakdev
Artifact	apijdbc
Name	apijdbc
Packaging	Jar
Java	21 o superior

Dependencias

Agregar únicamente:

- Spring Web
 - PostgreSQL Driver
-

Parte 2 — Crear la Base de Datos

Crear la base de datos:

```
CREATE DATABASE apijdbc;
```

Parte 3 — Crear la Tabla

Crear la siguiente tabla:

```
CREATE TABLE videojuegos(  
    codigo VARCHAR(10) PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL,  
    plataforma VARCHAR(50) NOT NULL,  
    precio DOUBLE NOT NULL,  
    disponible BOOLEAN NOT NULL,  
    genero VARCHAR(50)  
);
```

Explicación de la Tabla

Campo	Tipo
codigo	VARCHAR
nombre	VARCHAR
plataforma	VARCHAR
precio	DOUBLE
disponible	BOOLEAN
genero	VARCHAR

Parte 4 — Crear los Paquetes

Crear los siguientes paquetes:

```
com.krakdev.videojuegos.entidades  
com.krakdev.jdbc  
com.krakdev.jdbc.videojuegos  
com.krakdev.jdbc.videojuegos.services  
com.krakdev.jdbc.videojuegos.controller
```

Parte 5 — Crear la Entidad

Crear la clase:

Videojuego

Dentro del paquete:

`com.krakdev.videojuegos.entidades`

La entidad debe contener

Atributo	Tipo
codigo	String
nombre	String
plataforma	String
precio	double
disponible	boolean
genero	String

La clase debe incluir

- constructor vacío
 - constructor con parámetros
 - getters y setters
 - método `toString()`
-

Parte 6 — Clase Utilitaria de Conexión

Crear la clase:

Conexion.java

Dentro del paquete:

com.krakdev.jdbc

La clase debe

- conectarse a PostgreSQL
 - retornar Connection
 - manejar excepciones
 - utilizar logs
-

Pueden usar

- Logback
o
 - Log4j2
-

Parte 7 — Crear la Clase JDBC

Crear la clase:

VideojuegoJdbc

Dentro del paquete:

com.krakdev.jdbc.videojuegos

La clase debe implementar

Método insertar

Debe:

- usar INSERT
 - usar PreparedStatement
 - retornar un objeto Videojuego
-

Método listar

Debe:

- retornar List<Videojuego>
 - usar ResultSet
-

Método buscar

Debe:

- buscar por código
 - usar WHERE
-

Método actualizar

Debe:

- actualizar registros
 - retornar Videojuego actualizado
-

Método eliminar

Debe:

- eliminar registros
 - retornar boolean
-

Parte 8 — Crear la Capa Service

Crear la clase:

ServicioVideojuegoJdbc

Dentro del paquete:

com.krakdev.jdbc.videojuegos.services

La clase debe

- usar `@Service`
 - llamar los métodos JDBC
 - separar la lógica del controller
-

Métodos requeridos

Método	Retorno
crear	Videojuego
listar	List<Videojuego>
buscarPorCodigo	Videojuego
o	
actualizar	Videojuego
eliminar	boolean

Parte 9 — Crear el Controller

Crear la clase:

VideojuegoJdbcController

Dentro del paquete:

com.krakdev.jdbc.videojuegos.controller

La clase debe usar

@RestController
@RequestMapping

Endpoints requeridos

Método HTTP	Endpoint
POST	/jdbc/videojuegos
GET	/jdbc/videojuegos
GET	/jdbc/videojuegos/{codigo}
PUT	/jdbc/videojuegos/{codigo}
DELETE	/jdbc/videojuegos/{codigo}

Debe utilizar

- @PostMapping
 - @GetMapping
 - @PutMapping
 - @DeleteMapping
 - @PathVariable
 - @RequestBody
-

Parte 10 — Manejo de Excepciones

Todos los métodos deben:

- manejar excepciones
 - utilizar try-catch-finally
 - cerrar conexiones
 - mostrar logs
-

Parte 11 — Logs

El proyecto debe registrar mensajes como:

Conexión realizada

Videjuego insertado

Error al actualizar

Error al eliminar

Parte 12 — Probar con Postman

Probar POST

Enviar un JSON como:

```
{
  "codigo":"VG001",
  "nombre":"Minecraft",
  "plataforma":"PC",
  "precio":39.99,
  "disponible":true,
  "genero":"Sandbox"
}
```

Probar GET

Listar todos los videojuegos.

Probar GET por código

Ejemplo:

/jdbc/videojuegos/VG001

Probar PUT

Actualizar datos del videojuego.

Probar DELETE

Eliminar un videojuego.

Parte 13 — Evidencias para WhatsApp

Evidencia 1

Enviar captura mostrando:

- API ejecutándose
 - POST funcionando
-

Evidencia 2

Enviar captura mostrando:

- GET funcionando
 - respuesta JSON
-

Evidencia 3

Enviar captura mostrando:

- UPDATE o DELETE funcionando
-

Parte 15 — Commits Obligatorios

Commit	Descripción
1	Proyecto creado
2	Conexión PostgreSQL
3	Entidad creada
4	Insert funcionando
5	Listar funcionando
6	Buscar funcionando
7	Actualizar funcionando
8	Eliminar funcionando
9	API REST funcionando
10	Postman funcionando

Ejemplo

```
git add .  
git commit -m "Endpoint POST funcionando"
```

Parte 16 — Entrega Final

Subir el proyecto completo a GitHub.

La entrega será:

Link del repositorio GitHub

El repositorio debe contener

- código fuente
- commits
- estructura ordenada
- proyecto funcional